MPORA

Multi-Path Online Resource Allocation (MPORA) artifact for:

> **Uncertainty-Aware Resource Allocation for Multi-Path Programs with In-Kernel Predictions**
> Abigail Eisenklam, Carlos A. Montenegro G., Xian Wang, Yifan Cai, Robert Gifford, Linh Thi Xuan Phan, Ricardo G. Sanfelice, in ECRTS, 2026

This artifact trains and evaluates XGBoost models that predict two execution properties of SPEC CPU 2017 benchmarks — **remaining execution time** (`rem_time`) and **next instruction rate** (`next_insn`) — given lagged execution metrics obtained from profiling, the resource allocation, and features of the program's input. Trained models are exported to a compact Q16.16 fixed-point binary format and evaluated against a C inference library targeting low-overhead deployment inside of the kernel. Weighted conformal prediction (WCP) is then applied to produce statistically guaranteed prediction intervals, evaluated across execution-time scenarios.

Details for reproducing specific results in the paper can be found by searching this file for the corresponding figure or table, for example, using keyword FIGURE-1 or TABLE-1.

Requirements

Hardware

- NVIDIA GPU with drivers installed on the host (recommended for WCP weight network training; CPU fallback is automatic but slower)

Software

- Docker
- `nvidia-container-toolkit` (for GPU passthrough)

Setup

Pull the pre-built image (recommended, .csv files in `profiles/` exceed maximum size for Git):

```
docker pull abbyeisenklam/mpora-artifact:latest
docker run --gpus all -it abbyeisenklam/mpora-artifact:latest
```

Repository structure

```
MPORA/
├── profiles/                    # Input data (one subdirectory per
benchmark)
│   ├── mcf/
```

```
│   │   ├── normalized_data_delta=0.01.csv   # not in git — obtain
separately
│   │   ├── ...
│   │   ├── output_max_values_delta=0.01.pkl
│   │   └── input_max_values_delta=0.01.pkl
│   ├── omnetpp/
│   ├── xz/
│   ├── xalancbmk/
│   ├── exchange2/
│   └── x264/
│
├── c_implementation/             # Fixed-point C inference library
│   ├── xgb_utils_userspace.c
│   ├── xgb_utils_userspace.h
│   └── Makefile
│
├── kernel/
│   └── mpora.c                   # MPORA kernel module implementation
(corresponding to FIGURE-6, but view-only)
│
├── splits/                       # Pre-computed train/val/test/WCP splits
(per task)
│
├── models/                       # Output: trained models (created by
run.sh)
│   └── spec/<task>/delta=<d>/
│       ├── xgboost_model.json_target_0.model
│       ├── xgboost_model.json_target_1.model
│       ├── xgb_fp_target_0.bin  (+.meta.json)
│       └── xgb_fp_target_1.bin  (+.meta.json)
│
├── results/                      # Output: figures and metrics (created by
run.sh)
│   ├── spec/<task>/delta=<d>/
│   │   ├── results.txt
│   │   └── xgb_mae_histogram.pdf
│   ├── lineplot_nmae_rem_time.pdf
│   ├── lineplot_nmae_next_insn.pdf
│   └── wcp_results.txt
│
├── train.py                      # XGBoost training and evaluation script
├── train_utils.py                # Data loading, XGBoost training, C
evaluation
├── plotting_utils.py             # All plotting functions
├── plot_delta_vs_metric.py       # Aggregate NMAE-vs-Δ plot across
benchmarks
├── train_weight_net.py           # WCP weight network training and
coverage evaluation
├── train_weight_net.sh           # Convenience wrapper for
train_weight_net.py
├── cp.py                         # WeightedConformalPredictor
implementation
├── spec_input_feature_map.txt    # Per-benchmark input feature lists
```

```
├── requirements.txt
└── run.sh                          # Main runscript
```

Reproducing all results

```
./run.sh
```

This script:

1. Trains and evaluates XGBoost models for all six benchmarks × five Δ values
2. Plots per-task NMAE histograms and the aggregate NMAE vs. Δ line plots
3. Trains WCP weight networks for all benchmarks (Δ = 0.01) and evaluates per-scenario empirical coverage

Runtime of (1) and (2) combined is less than an hour on an Intel(R) Xeon(R) Silver 4216 CPU @ 2.10GHz.

Running a single XGBoost task

```
python train.py \
  --task mcf \
  --task-type spec \
  --window-size-delta 0.01 \
  --input-features
prev_budget,insn_sum,next_budget,prev_insn,inp_feat1,inp_feat2,prev_insn_1,
prev_insn_2,prev_insn_3,prev_budget_1,prev_budget_2,prev_budget_3 \
  --train-split 0.1 \
  --plot-nmae-histogram \
  --plot-train-runs
```

`train.py` arguments

Argument	Default	Description
`--task`	`mcf`	Benchmark name. Must match a subdirectory of `profiles/`.
`--task-type`	`spec`	Benchmark suite name. Used as a top-level directory under `models/` and `results/`.
`--csv`	*(derived)*	Path to `normalized_data_delta=<d>.csv`. Defaults to `profiles/<task>/normalized_data_delta=<d>.csv`.
`--input-features`	*(required)*	Comma-separated list of column names to use as model inputs.

Argument	Default	Description
`--train-split`	`0.1`	Fraction of distinct inputs to use for training. Remaining inputs are split equally into validation and test.
`--window-size-delta`	`0.01`	Observation window size Δ (in seconds) used during preprocessing. Selects which `normalized_data_delta=<d>.csv` file is loaded.
`--plot-nmae-histogram`	off	Save a per-target NMAE histogram PDF to `results/`.
`--plot-train-runs`	off	Save normalized profile plots for each training input to `profiles/<task>/`.

Running WCP separately

```
python train_weight_net.py           # all benchmarks
python train_weight_net.py --tasks mcf xz --verbose
bash train_weight_net.sh             # all benchmarks with default
hyperparameters
```

`train_weight_net.py` arguments

Argument	Default	Description
`--tasks`	*(all)*	Space-separated benchmark names to run.
`--n-clusters`	`4`	Number of execution-time scenarios.
`--delta`	`0.1`	Miscoverage level (target coverage = 1 − δ).
`--epochs`	`500`	Weight network training epochs.
`--lr`	`0.01`	Learning rate.
`--hidden-dims`	`64 32 8`	Hidden layer sizes of the weight network.
`--lambda_`	`1.0`	Entropy regularization weight.
`--beta`	`0.01`	L2 regularization strength.
`--batch-size`	`512`	Batch size for weight network training.
`--device`	`auto`	`auto`, `cuda`, or `cpu`.
`--output`	`results/wcp_results.txt`	Output file path.
`--verbose`	off	Print per-epoch training progress.

Input data

Each benchmark's `profiles/<task>/` directory contains one normalized CSV per Δ value. Each row is one observation window from one execution run.

Columns

Column	Description
`inp_name`	Input identifier (e.g. `input1`, `input42`)
`run_id`	Unique run identifier
`insn_sum`	Normalized cumulative instruction count (∈ [0, 1])
`prev_budget`	Resource budget allocated in the previous window (∈ [0, 1])
`next_budget`	Resource budget allocated in the next window (∈ [0, 1])
`prev_insn`	Instructions retired in the previous window (∈ [0, 1])
`inp_feat1`–`inp_feat6`	Benchmark-specific input features (see TABLE-1 for more details, ∈ [0, 1])
`prev_insn_1`–`prev_insn_N`	Lagged instruction rates (N varies per benchmark, ∈ [0, 1])
`prev_budget_1`–`prev_budget_N`	Lagged budget values (∈ [0, 1])
`rem_time`	**Target:** normalized remaining execution time (∈ [0, 1])
`next_insn`	**Target:** normalized next-window instruction rate (∈ [0, 1])

The `.pkl` files alongside each CSV store the per-feature max values used for normalization, and are loaded automatically during training.

Per-benchmark input features

The feature set used for each benchmark is defined in `spec_input_feature_map.txt` and passed to `train.py` via `--input-features` by `run.sh`.

Benchmark	Features
`mcf`	`prev_budget`, `insn_sum`, `next_budget`, `prev_insn`, `inp_feat1`, `inp_feat2`, `prev_insn_{1-3}`, `prev_budget_{1-3}`
`omnetpp`	`prev_budget`, `insn_sum`, `next_budget`, `prev_insn`, `inp_feat1`, `prev_insn_{1-4}`, `prev_budget_{1-4}`
`xz`	`prev_budget`, `insn_sum`, `next_budget`, `prev_insn`, `inp_feat1`, `inp_feat2`, `prev_insn_{1-3}`, `prev_budget_{1-3}`
`xalancbmk`	`prev_budget`, `insn_sum`, `next_budget`, `prev_insn`, `inp_feat1`, `prev_insn_{1-3}`, `prev_budget_{1-3}`
`exchange2`	`prev_budget`, `insn_sum`, `next_budget`, `prev_insn`, `inp_feat1`, `inp_feat6`, `prev_insn_{1-2}`, `prev_budget_{1-2}`

Benchmark	Features
x264	prev_budget, insn_sum, next_budget, prev_insn, inp_feat1–inp_feat4, prev_insn_{1-5}, prev_budget_{1-5}

Splits

Pre-computed input splits are stored in splits/<task>.txt (one line per role):

Role	Used for
train	XGBoost training
validation	XGBoost early stopping
test	All held-out inputs (union of the three below)
calibration	WCP calibration set D_cal
weights	WCP weight-network training set D_wt
test_cp	WCP coverage evaluation set D_test

Train inputs are selected by K-means clustering on execution time so that the training set spans the full range of input behaviors. The test inputs are split 40/40/20 into calibration/weights/test_cp.

Parameters tested

Parameter	Values
Window size Δ	0.01, 0.02, 0.03, 0.04, 0.05 (seconds)
Benchmarks	mcf, omnetpp, xz, xalancbmk, exchange2, x264
Train split	10% of distinct inputs
XGBoost trees	Up to 64 (early stopping, patience 20)
XGBoost depth	3
WCP miscoverage δ	0.1 (90% target coverage)
WCP scenarios	4 (quartile-binned by execution time)
Weight network	MLP 64→32→8, sigmoid output

Outputs

Per task, per Δ — results/spec/<task>/delta=<d>/

results.txt — XGBoost accuracy and C inference performance:

Column	Description
NMAE (float)	Normalized mean absolute error of the float XGBoost model
R^2 (float)	Coefficient of determination of the float model
NMAE (FP C)	NMAE of the Q16.16 fixed-point C inference
ΔNMAE	Accuracy gap between float and fixed-point (FP C − float)
Median Pred. Time (FP C)	Per-sample median inference time in µs (batch size 20, 10 repeats)
Model Size (FP C)	Size of the exported fixed-point binary

The files with these paths: `results/spec/<task>/delta=0.01/results.txt` reproduce the results that are shown in TABLE-2 for Δ = 0.01.

`xgb_mae_histogram.pdf` — histogram of per-sample absolute errors for both targets (`rem_time` and `next_insn`) from the float XGBoost model. This reproduces FIGURE-3 for `omnetpp` and Δ = 0.01, and generates the same style of figure for all other task and Δ combinations.

Aggregate — `results/`

`lineplot_nmae_rem_time.pdf`, **`lineplot_nmae_next_insn.pdf`** — NMAE vs. Δ for all six benchmarks on the `rem_time` and `next_insn` targets respectively. Generated by `plot_delta_vs_metric.py` after all Δ values have been evaluated. This reproduces FIGURE-2.

`wcp_results.txt` — WCP empirical coverage per (benchmark, scenario, target) at Δ = 0.01. Each scenario is a quartile of the test inputs sorted by execution time. Coverage should be ~ (1 − δ) = 90% in each scenario.

This file reproduces FIGURE-5 (looks like a table).

Column	Description
`task`	Benchmark name
`scenario`	Execution-time scenario index (0 = shortest, N−1 = longest)
`target`	`rem_time` or `next_insn`
`exec_range`	[min, max] normalized execution time of inputs in this scenario
`n_wt`	Number of weight-training samples (rows, not inputs)
`n_cal`	Number of calibration samples
`n_test`	Number of test samples used for coverage evaluation
`q`	Learned conformal threshold
`coverage`	Empirical coverage fraction
`target_coverage`	Target coverage (1 − δ)

Per task — `results/<task>/`

<task>_cluster_insn_sum_hist.pdf.png - stacked insn_sum histogram with one color per cluster (operating scenario) to show the distribution change between operating scenarios. Note that this only plots the distribution for one element of the state vector `x` (`insn_sum`). This will reproduce FIGURE-4.

Per task — `profiles/<task>/`

<input>_normalized_plot.png — normalized instruction rate vs. instruction sum profile for each training input, with static-budget runs highlighted in black and dynamic-budget runs shown semi-transparently in color. Generated only for Δ = 0.01 (pass `--plot-train-runs` to `train.py`). These figures show a more detailed version of FIGURE-1, for each benchmark and input in the training set.

Per task, per Δ — `models/spec/<task>/delta=<d>/`

File	Description
`xgboost_model.json_target_0.model`	Float XGBoost booster for `rem_time`
`xgboost_model.json_target_1.model`	Float XGBoost booster for `next_insn`
`xgb_fp_target_0.bin`	Fixed-point Q16.16 binary for `rem_time`
`xgb_fp_target_1.bin`	Fixed-point Q16.16 binary for `next_insn`
`*.bin.meta.json`	Feature order and input dimension metadata for each binary

MPORA Kernel Module

`kernel/mpora.c` shows the main decision making code that implements MPORA inside of the Linux kernel. The main resource allocation function, which is shown in `mpora_ml_try_allocate()`, solves the optimization problem in EQUATION-3 for N = 1 by leveraging the fixed-point C-converted XGBoost model. It reads each job's execution state and input features from the task metadata (in-tree Linux modification), predicts the remaining execution time and instruction rate under each resource allocation, and then keeps track of the optimal resource allcoation solution (which it then returns). This code is view-only, since it requires integration with a custom kernel and specialized hardware to execute as intended.